

Sistemas Operacionais

Aula 4: Escalonamento

Prof. Me. Emerson Ap. Mouco Jr.

Escalonamento

- Quando um computador é multiprogramado, ele frequentemente tem múltiplos processos ou threads competindo pela CPU ao mesmo tempo
- Essa situação ocorre sempre que dois ou mais deles estão simultaneamente no estado pronto

Escalonamento

- Se apenas uma CPU está disponível, uma escolha precisa ser feita sobre qual processo será executado em seguida
- A parte do sistema operacional que faz a escolha é chamada de escalonador, e o algoritmo que ele usa é chamado de algoritmo de escalonamento

Escalonamento

- Nos velhos tempos dos sistemas em lote com a entrada na forma de imagens de cartões em uma fita magnética, o algoritmo de escalonamento era simples: apenas execute o próximo trabalho na fita
- Com os sistemas de multiprogramação, ele tornou-se mais complexo porque geralmente havia múltiplos usuários esperando pelo serviço

Escalonamento

- Com a disseminação dos computadores pessoais, a situação mudou de duas maneiras:
- Primeiro, na maior parte do tempo há apenas um processo ativo. É improvável que um usuário preparando um documento em um processador de texto esteja simultaneamente compilando um programa em segundo plano

Escalonamento

- Quando o usuário digita um comando ao processador de texto, o escalonador não precisa fazer muito esforço para descobrir qual processo executar — o processador de texto é o único candidato
- Segundo, computadores tornaram-se tão rápidos com o passar dos anos que a CPU dificilmente ainda é um recurso escasso

Escalonamento

- A maioria dos programas para computadores pessoais é limitada pela taxa na qual o usuário pode apresentar a entrada (digitando ou clicando), não pela taxa na qual a CPU pode processá-la

Escalonamento

- Já em servidores de redes a conversa muda, múltiplos processos muitas vezes competem pela CPU, de maneira que o escalonamento seja importante
- Por exemplo, quando a CPU tem de escolher entre executar um processo que reúne as estatísticas diárias e um que serve a solicitações de usuários, estes ficarão muito mais contentes se o segundo receber a primeira chance de acessar a CPU

Escalonamento

- O argumento da “abundância de recursos” também não se sustenta em muitos dispositivos móveis, como smartphones (exceto talvez os modelos mais potentes)
- Além disso, a duração da bateria é uma das restrições mais importantes nesses dispositivos, alguns escalonadores tentam otimizar o consumo de energia

Escalonamento

- Dessa forma, além de escolher o processo certo a ser executado, o escalonador também tem de se preocupar em fazer um uso eficiente da CPU, pois o chaveamento de processos é algo caro (consome muita energia)

Escalonamento

- Quando é necessário escalonar?
- Na realidade, há uma série de situações nas quais o escalonamento é necessário. Primeiro, quando um novo processo é criado, uma decisão precisa ser tomada a respeito de qual processo, o pai ou o filho, deve ser executado

Escalonamento

- Tendo em vista que ambos os processos estão em um estado pronto, trata-se de uma decisão de escalonamento normal e pode ser qualquer uma, isto é, o escalonador pode legitimamente escolher executar o processo pai ou o filho em seguida

Escalonamento

- Segundo, uma decisão de escalonamento precisa ser tomada ao término de um processo
- Esse processo não pode mais executar (já que ele não existe mais), então algum outro precisa ser escolhido do conjunto de processos prontos. Se nenhum está pronto, um processo ocioso gerado pelo sistema normalmente é executado

Tipos de sistemas

- **Sistemas Interativos** são sistemas em que o usuário interage diretamente com o computador e recebe respostas quase imediatas
- Características principais:
 - Tempo de resposta rápido
 - Usuário dá comandos e vê o resultado logo
 - Fazem parte do nosso dia a dia

Tipos de sistemas

- **Exemplos de sistemas interativos:**
 - Sistemas bancários acessados pelo caixa eletrônico
 - Aplicativos de celular
 - Navegadores de internet
 - Jogos
 - Sistemas de atendimento ao cliente (chat online)
 - Linha de comando (quando você digita um comando e recebe resposta)

Tipos de sistemas

- **Sistemas em Lote (Batch)** são sistemas que processam tarefas em grandes blocos (lotes), sem interação direta do usuário durante a execução
- Características principais:
 - As tarefas são reunidas e executadas de uma vez
 - Não há resposta imediata
 - Normalmente usados para grandes volumes de dados
 - Normalmente são agendados (ex.: durante a madrugada)

Tipos de sistemas

- **Exemplos de sistemas em lote:**
 - Processamento de folha de pagamento
 - Geração de contas de luz/água
 - Processamento de relatórios diários ou mensais
 - Backups automáticos
 - Conversão de milhares de arquivos de uma só vez
 - Processos de mineração de dados

Tipos de sistemas

Características	Sistemas Interativos	Sistemas em Lote
Interação com usuário	Alta	Baixa (quase nenhuma)
Tempo de resposta	Imediato ou quase	Demorado
Quando é usado	Tarefas rápidas ou dinâmicas	Tarefas grandes e automáticas

Sistemas em lote

- **Primeiro a chegar, primeiro a ser servido (first-come, first-served)** é provavelmente o mais simples de todos os algoritmos de escalonamento
- Ness algoritmo, a CPU é atribuída aos processos na ordem em que a requisitam. Basicamente, há uma fila única de processos prontos

Sistemas em lote

- Quando a primeira tarefa entrar no sistema, ela é iniciada imediatamente e deixada executar por quanto tempo ela quiser
- Ela não é interrompida por ter sido executada por tempo demais. À medida que as outras tarefas chegam, elas são colocadas no fim da fila

Sistemas em lote

- Quando o processo que está sendo executado é bloqueado, o primeiro processo na fila é executado em seguida
- Quando um processo bloqueado fica pronto — assim como uma tarefa que chegou há pouco —, ele é colocado no fim da fila, atrás dos processos em espera

Sistemas em lote

- A grande vantagem desse algoritmo é que ele é fácil de compreender e fácil de programar
- Com esse algoritmo, uma única lista encadeada controla todos os processos. Escolher um processo para executar exige apenas remover um da frente da fila

Sistemas em lote

- Já a desvantagem desse algoritmo é que ele pode deixar a CPU travada com um processo pesado, enquanto outros processos mais rápidos, ficam esperando na fila para serem executados
- Uma forma de evitar isso seria usar preempção (interromper o processo pesado frequentemente) evita que ele atrapalhe os outros. Assim, processos rápidos terminam rápido, e o sistema fica mais eficiente

Sistemas em lote

- **Tarefa mais curta primeiro** assim como o anterior, esse também é um algoritmo em lote não preemptivo, mas neste caso ele presume que os tempos de execução são conhecidos antecipadamente
- Quando há vários processos igualmente importantes esperando na fila de entrada para serem iniciados, o escalonador escolhe a tarefa mais curta primeiro (**shortest job first**)

Sistemas em lote

8	4	4	4
A	B	C	D

Processos chegando

4	4	4	8
B	C	D	A

Ordenação do algoritmo

Sistemas em lote

- Vale a pena destacar que a tarefa mais curta primeiro é ótima apenas quando todas as tarefas estão disponíveis simultaneamente

Sistemas em lote

- **Tempo restante mais curto em seguida** uma versão preemptiva da tarefa mais curta primeiro é o tempo restante mais curto em seguida (**shortest remaining time next**)

Sistemas em lote

- **Tempo restante mais curto em seguida** uma versão preemptiva da tarefa mais curta primeiro é o tempo restante mais curto em seguida (**shortest remaining time next**)
- Com esse algoritmo, o escalonador escolhe o processo cujo tempo de execução restante é o mais curto. De novo, o tempo de execução precisa ser conhecido antecipadamente

Sistemas em lote

- Quando uma nova tarefa chega, seu tempo total é comparado com o tempo restante do processo atual
- Se a nova tarefa precisa de menos tempo para terminar do que o processo atual, este é suspenso e a nova tarefa iniciada. Esse esquema permite que tarefas curtas novas tenham um bom desempenho

Sistemas interativos

- **Escalonamento por chaveamento circular** um dos algoritmos mais antigos, simples, justos e amplamente usados é o circular (**round-robin**)
- A cada processo é designado um intervalo, chamado de seu quantum, durante o qual ele é deixado executar

Sistemas interativos

- Se o processo ainda está executando ao fim do quantum, a CPU sofrerá uma preempção e receberá outro processo
- Se o processo foi bloqueado ou terminado antes de o quantum ter decorrido, o chaveamento de CPU será feito quando o processo bloquear, é claro

Sistemas interativos

- O escalonamento circular é fácil de implementar. Tudo o que o escalonador precisa fazer é manter uma lista de processos executáveis. Quando o processo usa todo o seu quantum, ele é colocado no fim da lista.

Sistemas interativos

- **Escalonamento por prioridades** o escalonamento circular pressupõe implicitamente que todos os processos são de igual importância
- Por exemplo, em uma universidade uma ordem hierárquica começaria pelo reitor, os chefes de departamento em seguida, então os professores, secretários, zeladores e, por fim, os estudantes

Sistemas interativos

- A necessidade de levar em consideração fatores externos leva ao escalonamento por prioridades
- A ideia básica é direta: a cada processo é designada uma prioridade, e o processo executável com a prioridade mais alta é autorizado a executar

Sistemas interativos

- **Prioridade Estática** é atribuída ao processo no momento de sua criação e não muda durante a execução. Pode ser definida com base em:
 - **Classe do usuário** (ex.: administrador > professor > aluno)
 - **Tipo da tarefa** (ex.: processos do sistema > processos de usuário)
 - **Importância da aplicação** (ex.: serviços essenciais do sistema recebem prioridade alta)
- Essa abordagem é simples, mas tem o risco de fome (starvation): processos de baixa prioridade podem nunca ser executados

Sistemas interativos

- **Prioridade Dinâmica** aqui o sistema ajusta automaticamente a prioridade ao longo do tempo:
 - **Tempo de espera:** quanto mais tempo um processo fica esperando, maior sua prioridade (prevenção de starvation)
 - **Comportamento do processo:** processos que usam pouco a CPU → recebem prioridade maior (favorece I/O-bound) usam muita CPU → prioridade reduzida gradualmente (evita monopolização)

Sistemas interativos

- **Prioridade Dinâmica** aqui o sistema ajusta automaticamente a prioridade ao longo do tempo:
 - **Uso recente da CPU** - sistemas como Unix e Linux utilizam essa técnica: se o processo usou muita CPU recentemente, sua prioridade diminui
 - **Envelhecimento (aging)**: a prioridade aumenta com o tempo de espera. É uma forma de equilíbrio entre justiça e eficiência

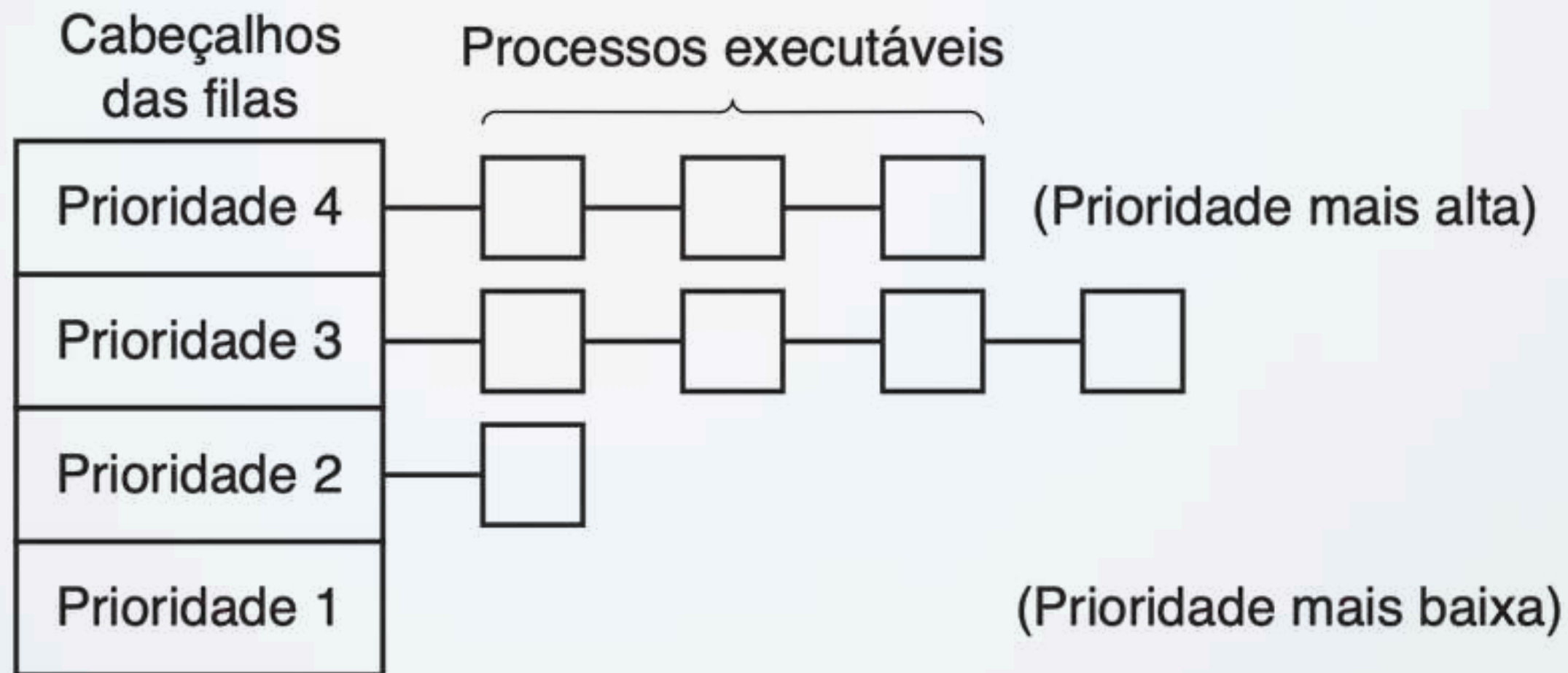
Sistemas interativos

- Para evitar que processos de prioridade mais alta executem indefinidamente, o escalonador talvez diminua a prioridade do processo que está sendo executado em cada tique do relógio (isto é, em cada interrupção do relógio)
- Se essa ação faz que a prioridade caia abaixo daquela do próximo processo com a prioridade mais alta, ocorre um chaveamento de processo

Sistemas interativos

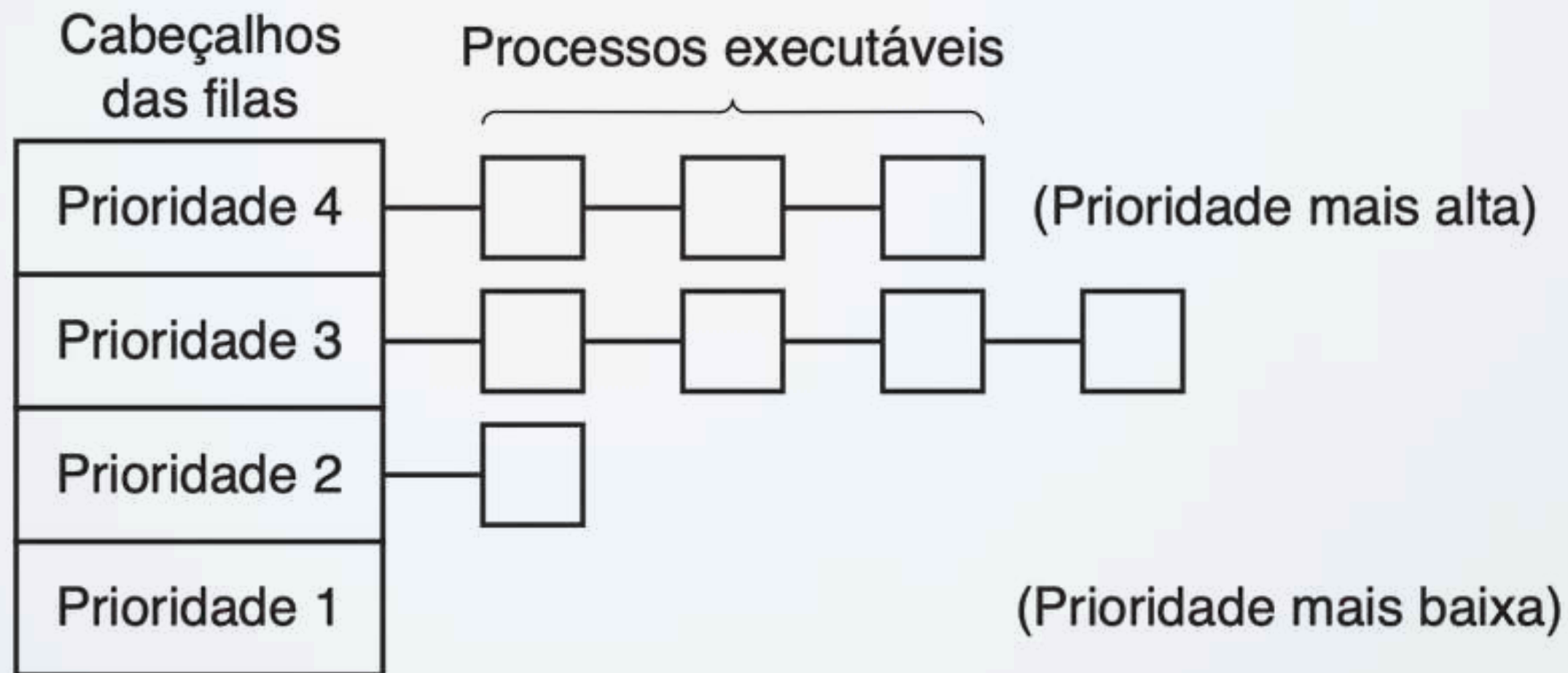
- Como alternativa, pode ser designado a cada processo um quantum de tempo máximo no qual ele é autorizado a executar
- Quando esse quantum for esgotado, o processo seguinte na escala de prioridade recebe uma chance de ser executado

Sistemas interativos



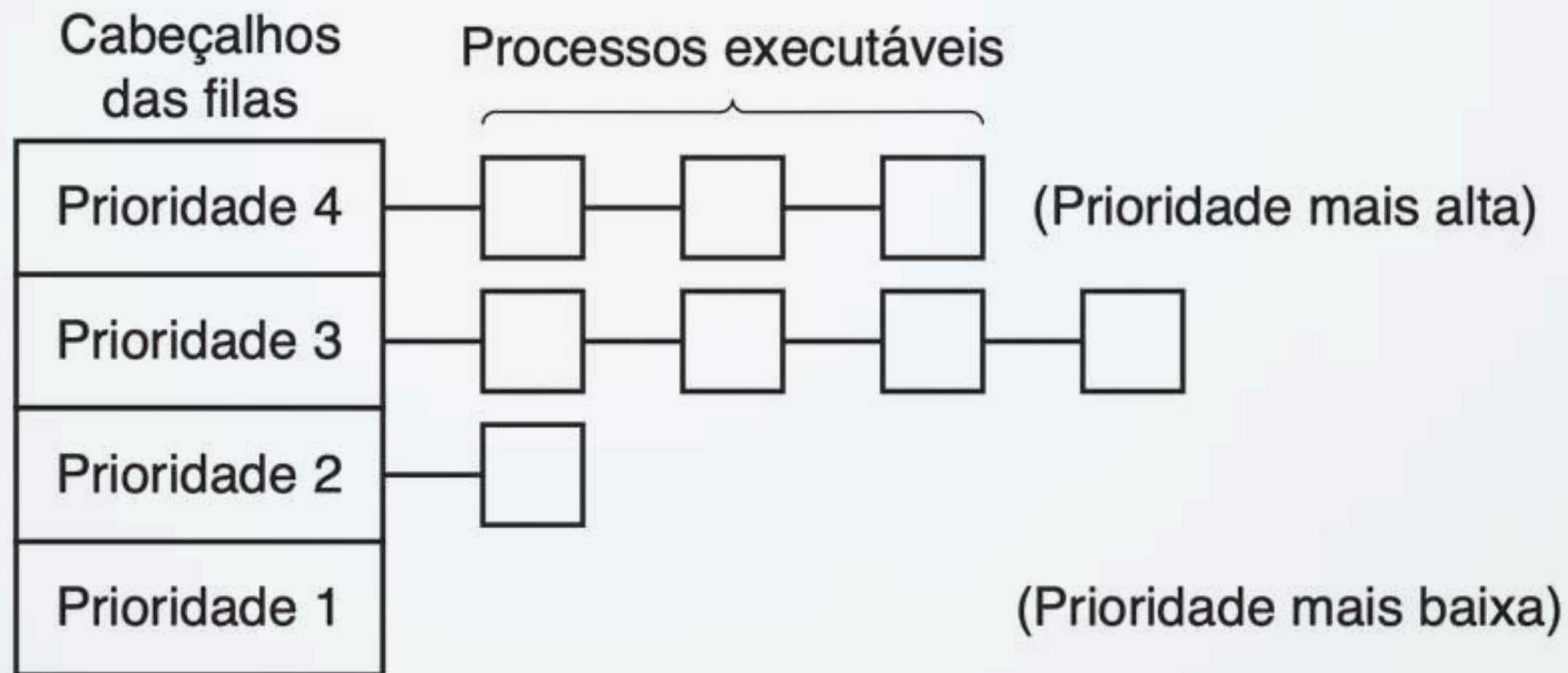
- Muitas vezes é conveniente agrupar processos em classes de prioridade e usar o escalonamento de prioridades entre as classes, mas escalonamento circular dentro de cada classe

Sistemas interativos



- A imagem mostra um sistema com quatro classes de prioridade. O algoritmo de escalonamento funciona do seguinte modo: desde que existam processos executáveis na classe de prioridade 4, apenas execute cada um por um quantum, estilo circular, e jamais se importe com classes de prioridade mais baixa

Sistemas interativos



- Se a classe de prioridade 4 estiver vazia, então execute os processos de classe 3 de maneira circular. Se ambas as classes — 4 e 3 — estiverem vazias, então execute a classe 2 de maneira circular e assim por diante

Escalonamento

Outras formas de escalonamento

- **Escalonamento garantido:** neste caso são feitas promessas reais para os usuários a respeito do desempenho. Por exemplo, se **n** usuários estão conectados enquanto você está trabalhando, você receberá em torno de **$1/n$** da potência da CPU. De modo similar, em um sistema de usuário único com n processos sendo executados, todos os fatores permanecendo os mesmos, cada um deve receber $1/n$ dos ciclos da CPU

Escalonamento

- **Escalonamento por loteria:** embora realizar promessas para os usuários e cumpri-las seja uma bela ideia, ela é difícil de implementar. No escalonamento por loteria, a ideia é dar "bilhetes de loteria" aos processos para vários recursos do sistema, como o tempo da CPU
- Sempre que uma decisão de escalonamento tiver de ser feita, um bilhete de loteria será escolhido ao acaso, e o processo com o bilhete fica com o recurso

Escalonamento

- Quando aplicado ao escalonamento de CPU, o sistema pode realizar um sorteio 50 vezes por segundo, com cada vencedor recebendo 20 ms de tempo da CPU como prêmio
- Parafraseando George Orwell: “Todos os processos são iguais, mas alguns processos são mais iguais” (TANENBAUM, 2015). Processos mais importantes podem receber bilhetes extras, para aumentar a chance de vencer

Escalonamento

- **Escalonamento por fração justa:** até agora presumimos que cada processo é escalonado por si próprio, sem levar em consideração quem o criou
- Se o usuário 1 inicia nove processos e o usuário 2 inicia um processo, parecidos ou com prioridades iguais, o usuário 1 receberá 90% da CPU e o usuário 2 apenas 10% dela

Escalonamento

- Para evitar essa situação, alguns sistemas levam em conta qual usuário é dono de um processo antes de escaloná-lo
- Nesse modelo, a cada usuário é alocada alguma fração da CPU e o escalonador escolhe processos de uma maneira que garanta essa fração. Desse modo, se dois usuários têm cada um 50% da CPU prometidos, cada um receberá isso, não importa quantos processos eles tenham em existência

Escalonamento

- Considere um sistema com dois usuários, cada um tendo a promessa de 50% da CPU, o usuário 1 tem quatro processos, **A, B, C e D**, e o usuário 2 tem apenas um processo, **E**

Escalonamento

- Considere um sistema com dois usuários, cada um tendo a promessa de 50% da CPU, o usuário 1 tem quatro processos, **A, B, C e D**, e o usuário 2 tem apenas um processo, **E**
- Se o escalonamento circular for usado, uma sequência de escalonamento possível que atende a todas as restrições é a seguinte:

A - E - B - E - C - E - D - E

Escalonamento

- Por outro lado, se o usuário 1 tem direito a duas vezes o tempo de CPU que o usuário 2, talvez tenhamos:

A B E C D E A B E C D E

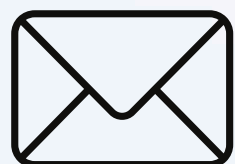
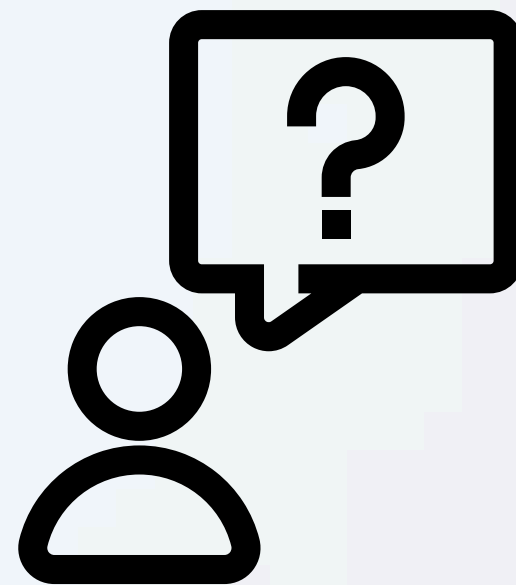
Escalonamento

- **Escalonamento em sistemas de tempo real:** um sistema de tempo real é aquele em que o tempo tem um papel essencial. Tipicamente, um ou mais dispositivos físicos externos ao computador geram estímulos, e o computador tem de reagir em conformidade dentro de um montante de tempo fixo
- Sistemas de tempo real estão monitorando pacientes em uma UTI, o piloto automático em um avião e o controle de robôs em uma fábrica automatizada. Em todos esses casos, ter a resposta certa, mas tê-la tarde demais é muitas vezes tão ruim quanto não tê-la

Escalonamento

- Sistemas em tempo real são geralmente categorizados como tempo real crítico, significando que há prazos absolutos que devem ser cumpridos e tempo real não crítico, significando que descumprir um prazo ocasional é indesejável, mas mesmo assim tolerável
- Os eventos a que um sistema de tempo real talvez tenha de responder podem ser categorizados ainda como **periódicos** (significando que eles ocorrem em intervalos regulares) ou **aperiódicos** (significando que eles ocorrem de maneira imprevisível)

Dúvidas?



emerson.mouco@cps.sp.gov.br



17 982236363